

Experiment 5 :

• Original Inserted Values:

1. hotel_post

	post_id	title	price	location
▶	1	Taj Palace	8000	Mumbai
	2	Sea View Resort	6500	Goa
	3	Mountain Retreat	5000	Manali
	4	City Lights Hotel	7000	Delhi
	5	Royal Heritage	9000	Jaipur
	6	Green Valley Stay	5500	Kerala
	7	Sunset Residency	6000	Pune
	8	Blue Lagoon Inn	7500	Chennai
*	NULL	NULL	NULL	NULL

2. company

	company_id	contact	name
▶	1	9619170233	Skyline Hospitality
	2	8369029538	Oceanic Group
	3	8454063683	Mountain View Pvt Ltd
	4	9612303854	Cityscape Hotels
	5	9823456712	Royal Stays
	6	9876543210	Green Leaf Resorts
	7	9123456780	Sunshine Hotels
	8	9988776655	BlueWave Hospitality
*	NULL	NULL	NULL

3. users

	user_id	role	first_name	last_name
▶	1	admin	Shivam	Pimple
	2	guest	Aryan	Patil
	3	guest	Sneha	Sharma
	4	admin	Radha	Ursekar
	5	guest	Neha	Kulkarni
	6	guest	Amit	Joshi
	7	guest	Priya	Desai
	8	guest	Kunal	Deshmukh
*	NULL	NULL	NULL	NULL

4. contacts

	user_id	contact
▶	1	9000011111
	2	9000022222
	3	9000033333
	4	9000044444
	5	9000055555
	6	9000066666
	7	9000077777
	8	9000088888
*	NULL	NULL

5. manages

	company_id	post_id
▶	1	1
	2	2
	3	3
	4	4
	5	5
	6	6
	7	7
	8	8
*	NULL	NULL

6. review

	review_id	post_id	user_id	rating
▶	6	6	7	2
	3	3	4	3
	2	2	3	4
	5	5	6	4
	8	8	2	4
	1	1	2	5
	4	4	5	5
	7	7	8	5
*	NULL	NULL	NULL	NULL

7. writes

	user_id	review_id
▶	2	1
	3	2
	4	3
	5	4
	6	5
	7	6
	8	7
	2	8
*	NULL	NULL

- **Implementation of single aggregate function at a time:**

- ❖ COUNT()

```
1 • SELECT
2     rating AS star_rating,
3     COUNT(*) AS total_ratings
4 FROM review
5 GROUP BY rating;
6
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	star_rating	total_ratings
▶	5	3
	4	3
	3	1
	2	1

Explanation:

This query counts the number of reviews for each rating by grouping all records based on the rating column.

- ❖ MAX()

```
1 • SELECT
2     location AS hotel_location,
3     MAX(price) AS maximum_price
4 FROM hotel_post
5 GROUP BY location;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	hotel_location	maximum_price
▶	Mumbai	8000
	Goa	6500
	Manali	5000
	Delhi	7000
	Jaipur	9000
	Kerala	5500
	Pune	6000
	Chennai	7500

Explanation:

This query finds the highest hotel price in each location by grouping records based on location.

❖ MIN()

```
1 • SELECT
2     location AS hotel_location,
3     MIN(price) AS minimum_price
4 FROM hotel_post
5 GROUP BY location;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	hotel_location	minimum_price
▶	Mumbai	8000
	Goa	6500
	Manali	5000
	Delhi	7000
	Jaipur	9000
	Kerala	5500
	Pune	6000
	Chennai	7500

Explanation:

This query finds the lowest hotel price in each location by grouping records based on location.

❖ AVG()

```
1 • SELECT
2     location AS hotel_location,
3     AVG(price) AS average_price
4 FROM hotel_post
5 GROUP BY location;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	hotel_location	average_price
▶	Mumbai	8000.0000
	Goa	6500.0000
	Manali	5000.0000
	Delhi	7000.0000
	Jaipur	9000.0000
	Kerala	5500.0000
	Pune	6000.0000
	Chennai	7500.0000

Explanation:

This query calculates the average hotel price for each location by grouping records based on location.

❖ SUM()

```
1 • SELECT
2     location AS hotel_location,
3     SUM(price) AS total_price
4 FROM hotel_post
5 GROUP BY location;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	hotel_location	total_price
▶	Mumbai	8000
	Goa	6500
	Manali	5000
	Delhi	7000
	Jaipur	9000
	Kerala	5500
	Pune	6000
	Chennai	7500

• Implementation of multiple aggregate functions at a time:

❖ SUM() & AVG()

```
1 • SELECT location AS hotel_location,
2     SUM(price) AS total_price,
3     AVG(price) AS average_price
4 FROM hotel_post
5 GROUP BY location;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	hotel_location	total_price	average_price
▶	Mumbai	8000	8000.0000
	Goa	6500	6500.0000
	Manali	5000	5000.0000
	Delhi	7000	7000.0000
	Jaipur	9000	9000.0000
	Kerala	5500	5500.0000
	Pune	6000	6000.0000
	Chennai	7500	7500.0000

Explanation:

This query calculates the total and average hotel price for each location by grouping records based on location.

❖ SUM() & COUNT()

```
1 • SELECT location AS hotel_location,  
2     SUM(price) AS total_price,  
3     COUNT(*) AS total_hotels  
4     FROM hotel_post  
5     GROUP BY location;
```

	hotel_location	total_price	total_hotels
►	Mumbai	8000	1
	Goa	6500	1
	Manali	5000	1
	Delhi	7000	1
	Jaipur	9000	1
	Kerala	5500	1
	Pune	6000	1
	Chennai	7500	1

Explanation:

This query calculates the total price and total number of hotels for each location by grouping records based on location

❖ SUM() & MAX()

```
1 • SELECT location AS hotel_location,  
2     SUM(price) AS total_price,  
3     MAX(price) AS maximum_price  
4     FROM hotel_post  
5     GROUP BY location;
```

	hotel_location	total_price	maximum_price
►	Mumbai	8000	8000
	Goa	6500	6500
	Manali	5000	5000
	Delhi	7000	7000
	Jaipur	9000	9000
	Kerala	5500	5500
	Pune	6000	6000
	Chennai	7500	7500

Explanation:

This query calculates the total price and highest hotel price for each location by grouping records based on location.

❖ SUM() & MIN()

```
1 • SELECT location AS hotel_location,
2     SUM(price) AS total_price,
3     MIN(price) AS minimum_price
4     FROM hotel_post
5     GROUP BY location;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	hotel_location	total_price	minimum_price
▶	Mumbai	8000	8000
	Goa	6500	6500
	Manali	5000	5000
	Delhi	7000	7000
	Jaipur	9000	9000
	Kerala	5500	5500
	Pune	6000	6000
	Chennai	7500	7500

Explanation:

This query calculates the total price and lowest hotel price for each location by grouping records based on location.

❖ AVG() & COUNT()

```
1 • SELECT location AS hotel_location,
2     AVG(price) AS average_price,
3     COUNT(*) AS total_hotels
4     FROM hotel_post
5     GROUP BY location;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	hotel_location	average_price	total_hotels
▶	Mumbai	8000.0000	1
	Goa	6500.0000	1
	Manali	5000.0000	1
	Delhi	7000.0000	1
	Jaipur	9000.0000	1
	Kerala	5500.0000	1
	Pune	6000.0000	1
	Chennai	7500.0000	1

Explanation:

This query calculates the average hotel price and total number of hotels for each location by grouping records based on location.

- **Implementation of HAVING clause:**

```
1 • SELECT location AS hotel_location,  
2     SUM(price) AS total_price,  
3     COUNT(*) AS total_hotels  
4     FROM hotel_post  
5     GROUP BY location  
6     HAVING SUM(price) > 6500;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	hotel_location	total_price	total_hotels
▶	Mumbai	8000	1
	Delhi	7000	1
	Jaipur	9000	1
	Chennai	7500	1

Explanation:

This query calculates total price and total hotels for each location and shows only those locations where the total price is greater than 15000.

```
1 • SELECT rating AS star_rating,  
2     COUNT(*) AS total_reviews  
3     FROM review  
4     GROUP BY rating  
5     HAVING COUNT(*) >= 2;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	star_rating	total_reviews
▶	5	3
	4	3

Explanation:

This query counts reviews for each rating and shows only those ratings that have two or more reviews.

- **Implementation of ORDER & LIMIT:**

```
1 • SELECT location AS hotel_location,  
2     SUM(price) AS total_price  
3     FROM hotel_post  
4     GROUP BY location  
5     ORDER BY total_price DESC  
6     LIMIT 1;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
hotel_location	total_price			
Jaipur	9000			

Explanation:

This query calculates the total hotel price for each location, sorts it in descending order, and shows only the location with the highest total price.

```
1 • SELECT rating AS star_rating,  
2     COUNT(*) AS total_reviews  
3     FROM review  
4     GROUP BY rating  
5     ORDER BY total_reviews ASC  
6     LIMIT 3;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:
star_rating	total_reviews			
3	1			
2	1			
5	3			

Explanation:

This query counts reviews for each rating, sorts them in ascending order, and shows the three ratings with the lowest number of reviews.

• Implementation of Nested Queries:

```
1 • SELECT location AS hotel_location,
2     COUNT(*) AS total_hotels
3     FROM hotel_post
4     WHERE price > (SELECT AVG(price) FROM hotel_post)
5     GROUP BY location;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	hotel_location	total_hotels
▶	Mumbai	1
	Delhi	1
	Jaipur	1
	Chennai	1

Explanation:

This query counts hotels in each location where the price is greater than the overall average hotel price using a nested subquery.

```
1 • SELECT rating AS star_rating,
2     COUNT(*) AS total_reviews
3     FROM review
4     WHERE rating < (SELECT AVG(rating) FROM review)
5     GROUP BY rating;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	star_rating	total_reviews
▶	3	1
	2	1

Explanation:

This query counts reviews for ratings that are greater than the overall average rating using a nested subquery.